

Bloque 10: Conexión a BBDD.

Este bloque corresponde a los **temas 18, 19 y 20** de la asignatura *Programación*.

1. Dónde guardo los datos en cada caso?	2
2. Conexión a bases de datos con JDBC	4
3. Sistemas objeto-relacional (Hibernate)	8
Anexo 1: comprobar instalación de MySQL	11
Anexo 2: cambiar contraseña de MySQL	12

Cuando desarrollamos aplicaciones en Java, en muchos casos necesitamos almacenar, consultar o modificar información de manera persistente. Si el volumen de información es elevado, necesita estar estructurado o relacionado, o no va a guardarse en local (si no en un servidor externo), será habitual conectar nuestros programas a bases de datos (como ya sabes, para datos más sencillos podemos simplemente guardarlos en ficheros `txt`, `xml`, `json`, etc.).

La forma tradicional de establecer esta comunicación es usando **JDBC** (Java Database Connectivity), una API¹ que permite ejecutar consultas SQL desde Java. Más adelante, para simplificar y automatizar parte de esta tarea, surgieron frameworks² como Hibernate, que permiten trabajar directamente con objetos Java sin necesidad de escribir consultas SQL de forma manual.

En la actualidad, conocer ambos enfoques es fundamental:

- **JDBC** para entender la base de la comunicación.
- **Hibernate** para programar de manera moderna, segura y eficiente.

1. Dónde guardo los datos en cada caso?

Es habitual tener dudas al principio con respecto a qué tipos de datos guardamos en memoria (variables, objetos, estructuras de datos), cuáles en ficheros y cuáles en bases de datos. Veamos una serie de ejemplos para intentar aclararlo:

#	Ejemplo	¿Dónde guardar?	Dato en memoria	Dato en disco	Justificación
1	Contador de intentos de login	Variable (<code>int</code>)	✓	✗	Solo se necesita mientras se ejecuta el programa.
2	Almacenar temporalmente el nombre y edad de un alumno introducidos por el usuario	Objeto (<code>Alumno</code>)	✓	✗	Datos relacionados, pero temporales.
3	Almacenar en memoria una lista de alumnos recién introducidos por consola	<code>ArrayList<Alumno></code>	✓	✗	Conjunto de objetos gestionados en tiempo de ejecución.
4	Guardar en memoria pares DNI → nombre para validación rápida durante el programa	<code>HashMap<String, String></code>	✓	✗	Ideal para búsquedas rápidas en memoria.
5	Configuración del programa (tema, idioma)	Fichero TXT	✗	✓	Pequeños datos que se guardan entre ejecuciones.
6	Lista de tareas con estado hecho/pendiente en una aplicación en local y sin usuarios	Fichero XML	✗	✓	Datos estructurados pero no complejos, sin necesidad de relaciones.
7	Historial que guarda en local la fecha y hora de cada acceso al programa	Fichero TXT	✗	✓	Un dato por línea, persistente pero básico.
8	Usuarios con contraseña, relacionado con más datos sobre el usuario	Base de datos	✗	✓	Requiere persistencia, control de duplicados, relaciones, consultas complejas...
9	Productos de una tienda	Base de datos	✗	✓	Relación entre productos, precios, stock, consultas complejas...

#	Ejemplo	¿Dónde guardar?	Dato en memoria	Dato en disco	Justificación
10	Plataforma educativa con alumnos, cursos, profesores	Base de datos	✗	✓	Persistencia, muchos datos relacionados, consultas complejas...

No obstante, es importante destacar que en la mayoría de ocasiones se necesitarán datos persistentes y en memoria.

Ejemplo práctico: en el caso 10, plataforma educativa, los datos de alumnos, profesores y cursos deberán estar guardados en una base de datos. Sin embargo, si hay una pantalla donde el profesor pueda ver su lista de alumnos, lo más probable es que se consulte a la base de datos los alumnos relacionados con ese profesor y se carguen los resultados en una estructura de datos, como un ArrayList o similar, previamente a mostrarse en la página.

2. Conexión a bases de datos con JDBC

JDBC es una API¹ de Java que proporciona las herramientas necesarias para:

- Conectarse a bases de datos.
- Ejecutar consultas SQL (INSERT, SELECT, UPDATE, DELETE).
- Procesar los resultados.

Pasos para conectarse e interactuar con una bbdd en JDBC:

1. Necesitamos tener el servidor de MySQL ejecutándose en nuestro equipo, y conocer el puerto (normalmente 3306), el usuario (normalmente root) y la contraseña.
2. **Cargar el driver JDBC.** En función del gestor de bases de datos al que nos vayamos a conectar (MySQL, SQL Server, PostgreSQL, Oracle, etc.), deberemos instalar el driver correspondiente. Existen varias formas de añadir el driver JDBC a nuestro proyecto de Java:
 - Descargarlo manualmente de la web oficial y añadirlo manualmente a la carpeta de dependencias del proyecto.
 - Si nuestro proyecto está creado con Maven o Gradle, podemos descargarlo directamente desde su repositorio.
3. Crear la programación de Java que deberá crear una conexión (**Connection**), crear y ejecutar una consulta (**Statement** o **PreparedStatement**), procesar los resultados (**ResultSet**), cerrar la conexión y recursos y controlar las excepciones de sql.

Principales elementos de una conexión con bbdd:

- **Connection**: clase que nos permite realizar la conexión con la bbdd.
- **Statement**: Permite ejecutar consultas SQL simples (más básica).
- **PreparedStatement**: Permite consultas que internamente son más seguras y eficientes.
- **ResultSet**: funciona como un cursor o puntero que va apuntando fila a fila de los resultados devueltos para poder leerlos. Su método **next()** avanza a la siguiente fila y devuelve **true** si hay más datos o **false** si no. Después, los métodos **getInt("edad")** o **getString("nombre")** leen los campos de dicha fila especificándole la columna.
- **SQLException**: excepción que indica un error relacionado con la bbdd (error en la sintaxis de la consulta, pérdida de la conexión con la bbdd, etc.).

Importante sobre los métodos de ejecución:

- Si la consulta SQL **devuelve datos** (por ejemplo, un **SELECT**), se debe usar **executeQuery()**, que devuelve un **ResultSet** para recorrer los resultados.
- Si la consulta SQL **no devuelve resultados** (por ejemplo, un **INSERT**, **UPDATE**, **DELETE**), se debe usar **executeUpdate()**, que devuelve un número indicando las filas afectadas.
- **ResultSet** solo se utiliza en consultas que devuelven información (**SELECT**). Para las demás operaciones (**INSERT**, **UPDATE**, **DELETE**) no es necesario.

Ejemplo 1: lectura con **executeQuery()** y resultados con **ResultSet**

```
Java
String url = "jdbc:mysql://localhost:3306/instituto";
String user = "root";
String password = "admin123";

String sqlSelect = "SELECT * FROM instituto";

try {
    Connection conn = DriverManager.getConnection(url, user, password);
    PreparedStatement stmt = conn.prepareStatement(sqlSelect);

    ResultSet rs = stmt.executeQuery();

    while (rs.next()) {
        int id = rs.getInt("id");
        String nombre = rs.getString("nombre");
        System.out.println(id + ": " + nombre);
    }

    rs.close();
    stmt.close();
    conn.close();

} catch (SQLException e) {
    System.out.println("Error de SQL " + e.getMessage() );
} catch (Exception e) {
    System.out.println("Error desconocido" + e.getMessage() );
}
```

Ejemplo 2: inserción con `executeUpdate()`

```
Java
String url = "jdbc:mysql://localhost:3306/instituto";
String user = "root";
String password = "admin123";

String sqlInsert = "INSERT INTO alumnos (nombre, nota_media) VALUES (?, ?)";

try {
    Connection conn = DriverManager.getConnection(url, user, password);
    PreparedStatement pstmt = conn.prepareStatement(sqlInsert);

    pstmt.setString(1, "Ana Garcia");
    pstmt.setDouble(2, 8.5);

    int filasInsertadas = pstmt.executeUpdate();

    System.out.println("Insertados: " + filasInsertadas + " registros.");

    pstmt.close();
    conn.close();

} catch (SQLException sqle) {
    System.out.println("Error de SQL al insertar: " + sqle.getMessage());
} catch (Exception e) {
    System.out.println("Error desconocido: " + e.getMessage());
}
```

Ejemplo 3: actualización con `executeUpdate()`

```
Java
String url = "jdbc:mysql://localhost:3306/instituto";
String user = "root";
String password = "admin123";

String sqlUpdate = "UPDATE alumnos SET curso = ? WHERE nombre = ?";

try {
    Connection conn = DriverManager.getConnection(url, user, password);
    PreparedStatement pstmt = conn.prepareStatement(sqlUpdate);

    pstmt.setString(1, "2DAM");
    pstmt.setString(2, "Ana Garcia");

    int filasActualizadas = pstmt.executeUpdate();

    System.out.println("Actualizados: " + filasActualizadas + " registros");

    pstmt.close();
    conn.close();

} catch (SQLException sqle) {
    System.out.println("Error de SQL al actualizar: " + sqle.getMessage());
} catch (Exception e) {
    System.out.println("Error desconocido: " + e.getMessage());
}
```

Ejemplo 4: borrado con `executeUpdate()`

Java

```
String url = "jdbc:mysql://localhost:3306/instituto";
String user = "root";
String password = "admin123";

String sqlDelete = "DELETE FROM alumnos WHERE nombre = ?";

try {
    Connection conn = DriverManager.getConnection(url, user, password);
    PreparedStatement pstmt = conn.prepareStatement(sqlDelete);

    pstmt.setString(1, "Ana Garcia");
    int filasBorradas = pstmt.executeUpdate();

    System.out.println("Borrados: " + filasBorradas + " registros");

    pstmt.close();
    conn.close();

} catch (SQLException sqle) {
    System.out.println("Error de SQL al borrar: " + sqle.getMessage());
} catch (Exception e) {
    System.out.println("Error desconocido: " + e.getMessage());
}
```


3. Sistemas objeto-relacional (Hibernate)

Como ya sabes, en programación lo ideal es trabajar con Programación Orientada a Objetos. Sin embargo, las bases de datos trabajan con tablas (filas y columnas). Para conectar estos dos mundos se utilizan sistemas de mapeo objeto-relacional (ORM), que permiten asociar clases y atributos de Java con tablas y columnas de una base de datos.

Existen varios sistemas de este tipo, uno de los más conocidos es **Hibernate**.

Hibernate es un framework² que permite:

- Mapear (convertir) la información entre bases de datos relacionales y objetos de Java.
- Gestionar conexiones, inserciones, consultas y actualizaciones de manera automática, sin necesidad de escribir sentencias SQL manualmente.
- Reduce errores al automatizar la conversión entre objetos y registros.
- Permite trabajar con múltiples gestores de bases de datos sin cambiar el código.
- Mejora el mantenimiento y escalabilidad del código.

Conceptos clave en Hibernate:

- **@Entity**: Clase Java que representa una tabla (entidad) de la base de datos. Sus atributos se corresponden con las columnas.
- **@Id**: Identifica la clave primaria de la entidad.
- **Session**: Objeto que gestiona la conexión entre Java y la base de datos.
- **SessionFactory**: Se encarga de crear sesiones (conexiones).
- **Transaction**: Permite gestionar las transacciones (confirmar o deshacer, por ejemplo).
- Operaciones **CRUD** (crear, leer, actualizar, borrar):
 - **save()**: Inserta un nuevo objeto en la base de datos.
 - **get()**: Recupera un objeto desde la base de datos utilizando su clave primaria.
 - **update()**: Actualiza los valores de un objeto ya existente en la base de datos.
 - **delete()**: Elimina un objeto de la base de datos.

Ejemplo básico: crear una clase y definirla como entidad.

```
Java
@Entity // Indica que esta clase se corresponde con una tabla
@Table(name = "alumno") // Nombre de la tabla en la base de datos
public class Alumno {

    @Id //Indica que este atributo será la clave primaria
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Indica que será una
clave primaria autoincremental
    private int id;

    private String nombre;
    private String email;

    //Getters y Setteres
}
```

Ejemplo básico: establecer conexión.

```
Java
SessionFactory factory = new Configuration()
    .configure("hibernate.cfg.xml") // Carga la configuración desde el archivo
XML (ahí estará la dirección, usuario, contraseña, etc.).
    .addAnnotatedClass(Alumno.class) // Registra la clase mapeada
    .buildSessionFactory();

Session session = factory.getCurrentSession(); // Abre una sesión (conexión)
```

Ejemplo básico: guardar un objeto en una tabla.

```
Java
session.beginTransaction();
session.save(alumno); // INSERT automático en la tabla alumno
session.getTransaction().commit();
```

Ejemplo básico: leer un objeto de la tabla y crear un objeto.

```
Java
Alumno alumno = session.get(Alumno.class, 1); // SELECT automático donde id = 1
```

¹ **API** Una API (Application Programming Interface) es un conjunto de métodos o reglas que permiten que un programa se comunique con otro módulo, componente o sistema (en este caso, con el servidor de la base de datos).

² **Framework** Un framework es un conjunto de herramientas que proporciona una estructura base sobre la que se construyen aplicaciones. Incluye librerías (clases y métodos ya creados) y exige seguir una forma concreta de organizar el código.

Anexo 1: comprobar instalación de MySQL

Para comprobar si tenemos MySQL correctamente instalado, debemos acceder a **C:\Program Files\MySQL** y comprobar que tenemos, como mínimo, las siguientes carpetas:

- MySQL Server
- MySQL Workbench

Si falta alguno, debemos descargar la última versión de:
<https://dev.mysql.com/downloads/>

Anexo 2: cambiar contraseña de MySQL

Si no recuerdas tu contraseña para acceder al servicios MySQL sigue los siguientes pasos:

1. Detener el servicio de MySQL: antes de hacer cualquier cambio, el servidor debe estar apagado.

- Presiona **Win + R**, escribe **services.msc** y pulsa Enter.
- Busca el servicio llamado **MySQL** o **MySQL80**.
- Haz clic derecho y selecciona **Detener**.

2. Crear un archivo de inicialización: vamos a engañar un poco al sistema para que acepte una nueva contraseña al arrancar.

1. Abre el Bloc de Notas.
2. Pega la siguiente línea (reemplaza **TuNuevaContraseña** por la que quieras):

```
ALTER USER 'root'@'localhost' IDENTIFIED BY  
'TuNuevaContraseña';
```

3. Guarda el archivo en la raíz del disco **C:** con el nombre **mysql-init.txt**.

3. Ejecutar MySQL en modo de recuperación: ahora necesitamos arrancar MySQL indicándole que lea ese archivo.

1. Busca **"CMD"** en el inicio, haz clic derecho y selecciona **Ejecutar como administrador**.
2. Navega hasta la carpeta **bin** de tu instalación de MySQL. Generalmente es algo como: `cd "C:\Program Files\MySQL\MySQL Server 8.0\bin"`
3. Ejecuta el siguiente comando (ajusta la ruta del archivo si la cambiaste):
`mysqld --init-file=C:\mysql-init.txt`

4. Limpieza y Reinicio

- Si el comando parece "quedarse pegado" o no devuelve nada después de unos segundos, suele ser buena señal. Espera un momento y cierra esa ventana del CMD.
- Vuelve a la ventana de **Servicios (services.msc)** y dale a **Iniciar** al servicio de MySQL.
- **Importante:** Borra el archivo **C:\mysql-init.txt** por seguridad, ya que contiene tu contraseña en texto plano.

